

Theoretical overview: Deep infinitely wide fully connected Neural network

Report by: Rezoanoor Rahman

April 2026

1 Introduction

The paper “Deep Neural Networks as Gaussian Processes” [Lee et al., 2018] formalize the connection between deep neural networks and Gaussian processes in the infinite-width limit. While Neal [1996] work established this equivalence for single-layer networks, the authors extend it to deep architectures, showing that as each hidden layer becomes infinitely wide, the network output converges to a Gaussian process.

The key analytical contribution is a recursive formulation of the kernel. Using the Central Limit Theorem, the authors argue that pre-activations at each layer become Gaussian, and this property propagates through depth. As a result, the covariance function $K^L(x, x)$ of the output can be computed layer-by-layer using a deterministic recursion that depends only on the activation function and hyperparameters such as weight and bias variances.

The major challenge is evaluating this recursion efficiently for general nonlinearities. For activations like *ReLU*, closed-form expressions exist, but for others (e.g., *Tanh*), the required Gaussian expectations needed to be computed numerically. The paper introduces a lookup table method, where these expectations are precomputed over a grid and reused via interpolation, significantly reducing computational cost.

With this kernel, the authors perform exact Bayesian inference using standard GP formulas, avoiding gradient-based training. Empirical results show that these NNGP models achieve performance comparable to trained neural networks, and that finite networks approach this behavior as width increases.

In this report, we first summarize the theoretical and empirical results from the reference article. Then we implemented the process mentioned in the article using `PyTorch`. In addition to that, we made some tweaking in the training process to speed up the lookup interpolation method, which is described in section ?? . Finally, in section 5, we provided our comments on several factors of the article.

2 Methods

In our report, we primarily follow the notation used by Lee et al. [2018]. Here, we consider an L -hidden-layer fully-connected neural network with hidden layers of width N_l (for layer l) and pointwise nonlinearities ϕ . Let $x \in \mathbb{R}^{d_{\text{in}}}$ and $z^L \in \mathbb{R}^{d_{\text{out}}}$ denote the input and output, respectively. The i th post- and pre-activations in the l th layer are x_i^l and z_i^l . Note that, we have $x_i^0 \equiv x_i$ for the input. Weight and bias parameters for the l th layer have components W_{ij}^l, b_i^l , which are independent and randomly drawn from $N(0, \sigma_w^2/N_l)$ and $N(0, \sigma_b^2)$.

2.1 Review of Gaussian Processes and Single-Layer Neural Networks

The correspondence between single-hidden layer neural networks and GPs were developed by Neal [1996]. For a fully connected single layer neural network we can write the i th output as

$$z_i^1(x) = b_i^1 + \sum_{j=1}^{N_1} W_{ij}^1 x_j^1(x), \quad x_j^1(x) = \phi\left(b_j^0 + \sum_{k=1}^{d_{\text{in}}} W_{jk}^0 x_k\right), \quad (1)$$

Because the weight and bias parameters are taken to be i.i.d., the post-activations $x_j^1, x_{j'}^1$ are independent for $j \neq j'$. Moreover, since $z_i^1(x)$ is a sum of i.i.d. terms, it follows from the Central Limit Theorem that in the limit of infinite width $N_1 \rightarrow \infty$, $z_i^1(x)$ will be Gaussian distributed. Likewise, from the multidimensional Central Limit Theorem, any finite collection of $\{z_i^1(x^{\alpha=1}), \dots, z_i^1(x^{\alpha=k})\}$ will have a joint multivariate Gaussian distribution, which is exactly the definition of a Gaussian process. Therefore we conclude that $z_i^1 \sim \mathcal{GP}(\mu^1, K^1)$, a GP with mean μ^1 and covariance K^1 , which are themselves independent of i . Because the parameters have zero mean, we have that $\mu^1(x) = \mathbb{E}[z_i^1(x)] = 0$ and,

$$K^1(x, x') \equiv \mathbb{E}[z_i^1(x) z_i^1(x')] = \sigma_b^2 + \sigma_w^2 \mathbb{E}[x_i^1(x) x_i^1(x')] = \sigma_b^2 + \sigma_w^2 C(x, x'), \quad (2)$$

where $C(u, u') = \mathbb{E}_{(u, u') \sim \mathcal{N}(0, \Sigma)} [\phi(u) \phi(u')]$

This expression is highly dependent of the choice of activation function $\phi(\cdot)$. For $\phi(x) = \text{ReLU}(x)$, a closed form is derived by Cho and Saul [2009], which is

$$C(x, x') = \frac{\sigma_w^2}{2\pi} \sqrt{K(x, x) K(x', x')} (\sin \theta_{x, x'} + (\pi - \theta_{x, x'}) \cos \theta_{x, x'}), \quad (3)$$

$$\theta_{x, x'} = \cos^{-1} \left(\frac{K(x, x')}{\sqrt{K(x, x) K(x', x')}} \right). \quad (4)$$

However, for other activation functions, a closed form is not available, and hence numerical computation is necessary. For example, if $\phi(x) = \tanh(x)$, we can represent $K()$ as an infinite sum, provided in appendix A, which is practically not appealing.

2.2 Gaussian Processes and Deep Neural Networks

The same argument has been extended for fully connected deep neural network by Lee et al. [2018]. The arguments of the previous section can be extended to deeper layers by induction by taking the hidden layer widths to be infinite in succession ($N_1 \rightarrow \infty, N_2 \rightarrow \infty$, etc.). As $N_1 \rightarrow \infty$, the baseline condition is true by the previous section.

Now, suppose that z_j^{l-1} is a GP, identical and independent for every j (and hence $x_j^l(x)$ are i.i.d.). After $l-1$ steps, we have

$$z_i^l(x) = b_i^l + \sum_{j=1}^{N_l} W_{ij}^l x_j^l(x), \quad x_j^l(x) = \phi\left(z_j^{l-1}(x)\right). \quad (5)$$

Since $z_i^l(x)$ is a sum of i.i.d. random terms so that, as $N_l \rightarrow \infty$, any finite collection $\{z_i^l(x^{\alpha=1}), \dots, z_i^l(x^{\alpha=k})\}$ will have joint multivariate Gaussian distribution and $z_i^l \sim \mathcal{GP}(0, K^l)$. The covariance is

$$K^l(x, x') = \mathbb{E}[z_i^l(x) z_i^l(x')] = \sigma_b^2 + \sigma_w^2 \mathbb{E}_{z_i^{l-1} \sim \mathcal{GP}(0, K^{l-1})} \left[\phi\left(z_i^{l-1}(x)\right) \phi\left(z_i^{l-1}(x')\right) \right]. \quad (6)$$

By induction, the expectation in Equation 6 is over the GP governing z_i^{l-1} , but this is equivalent to integrating against the joint distribution of only $z_i^{l-1}(x)$ and $z_i^{l-1}(x')$. The latter is described by a zero mean, two-dimensional Gaussian whose covariance matrix has distinct entries $K^{l-1}(x, x')$, $K^{l-1}(x, x)$, and $K^{l-1}(x', x')$. Hence, we can write

$$K^l(x, x') = \sigma_b^2 + \sigma_w^2 F_\phi \left(K^{l-1}(x, x'); K^{l-1}(x, x), K^{l-1}(x', x') \right) \quad (7)$$

to emphasize the recursive relationship between K^l and K^{l-1} via a deterministic function F whose form depends only on the nonlinearity ϕ .

For the base case K^0 , suppose $W_{ij}^0 \sim \mathcal{N}(0, \sigma_w^2/d_{\text{in}})$, $b_j^0 \sim \mathcal{N}(0, \sigma_b^2)$; we can utilize the recursion relating K^1 and K^0 , where

$$K^0(x, x') = \mathbb{E}[z_j^0(x)z_j^0(x')] = \sigma_b^2 + \sigma_w^2 \left(\frac{x \cdot x'}{d_{\text{in}}} \right). \quad (8)$$

2.3 Bayesian Training of Neural Networks using Gaussian Process Priors

Given a dataset $\mathcal{D} = \{(x^1, t^1), \dots, (x^n, t^n)\}$, we wish to make a Bayesian prediction at test point x^* using a distribution over functions $z(x)$. This distribution is constrained to take values $z \equiv (z^1, \dots, z^n)$ on the training inputs $x \equiv (x^1, \dots, x^n)$ and,

$$P(z^* | \mathcal{D}, x^*) = \int P(z^* | z, x, x^*) P(z | \mathcal{D}) dz = \frac{1}{P(t)} \int P(z^*, z | x^*, x) P(t | z) dz, \quad (9)$$

where $t = (t^1, \dots, t^n)^T$ are the targets on the training set, and $P(t | z)$ corresponds to observation noise. Here, it is assumed $P(t | z) \stackrel{d}{=} N(z, \sigma_\epsilon^2)$. Using MVN theory, we can say $z^*, z | x^*, x \sim \mathcal{N}(0, \mathbf{K})$ with

$$\mathbf{K} = \begin{bmatrix} K_{\mathcal{D}, \mathcal{D}} & K_{x^*, \mathcal{D}}^T \\ K_{x^*, \mathcal{D}} & K_{x^*, x^*} \end{bmatrix},$$

Here, $K_{\mathcal{D}, \mathcal{D}}$ is an $n \times n$ matrix whose (i, j) th element is $K(x^i, x^j)$ with $x^i, x^j \in \mathcal{D}$, while e.g. the i th element of $K_{x^*, \mathcal{D}}$ is $K(x^*, x^i)$, $x^i \in \mathcal{D}$. As is standard in GPs, the integral in Equation 9 can be done exactly, resulting in $z^* | \mathcal{D}, x^* \sim \mathcal{N}(\bar{\mu}, \bar{K})$ with

$$\bar{\mu} = K_{x^*, \mathcal{D}}(K_{\mathcal{D}, \mathcal{D}} + \sigma_\epsilon^2 I_n)^{-1} t, \quad (10)$$

$$\bar{K} = K_{x^*, x^*} - K_{x^*, \mathcal{D}}(K_{\mathcal{D}, \mathcal{D}} + \sigma_\epsilon^2 I_n)^{-1} K_{x^*, \mathcal{D}}^T, \quad (11)$$

where I_n is the $n \times n$ identity. The predictive distribution $z^* | \mathcal{D}, x^*$ is obtained via standard GP matrix computations, yet corresponds to fully Bayesian training of the deep neural network. The covariance K^L is determined by the GP prior induced by the network architecture, depending on depth, nonlinearity, and weight and bias variances.

2.4 Implementation of the GP Kernel

As previously mentioned, for an L -layer neural network with fixed hyperparameters, the GP kernel K^L can be obtained by recursively evaluating the Gaussian expectation in Equation 7 for all input pairs (x, x') . For ReLU activation, this expectation can be computed analytically; for general nonlinearities, numerical evaluation is required.

A straightforward implementation computes the integral separately for every pair of datapoints at every layer; has computational cost $\mathcal{O}(n_g^2 L(n_{\text{train}}^2 + n_{\text{train}} n_{\text{test}}))$, which is impractical, even for moderate dataset sizes.

In this context, Lee et al. [2018] suggested to do the following steps:

1. Normalize inputs so that all x have equal norm. This ensures $K^{l-1}(x, x) = K^{l-1}(x', x')$, so the recursion depends only on marginal variance s and correlation c .
2. Construct fixed grids:

$$u = [-u_{\max}, \dots, u_{\max}], \quad s = [0, \dots, s_{\max}], \quad c = [-1, \dots, 1]$$

with sizes n_g , n_v , and n_c , respectively.

3. Build a lookup table $F \in \mathbb{R}^{n_v \times n_c}$ approximating F_ϕ (Eq. 7):

$$F_{ij} = \frac{\sum_{ab} \phi(u_a) \phi(u_b) \exp\left(-\frac{1}{2} \begin{bmatrix} u_a \\ u_b \end{bmatrix}^T \begin{bmatrix} s_i & s_i c_j \\ s_i c_j & s_i \end{bmatrix}^{-1} \begin{bmatrix} u_a \\ u_b \end{bmatrix}\right)}{\sum_{ab} \exp\left(-\frac{1}{2} \begin{bmatrix} u_a \\ u_b \end{bmatrix}^T \begin{bmatrix} s_i & s_i c_j \\ s_i c_j & s_i \end{bmatrix}^{-1} \begin{bmatrix} u_a \\ u_b \end{bmatrix}\right)}. \quad (12)$$

4. For each layer l and pair (x, x') , compute:

$$s = K^{l-1}(x, x), \quad c = \frac{K^{l-1}(x, x')}{K^{l-1}(x, x)}.$$

5. Obtain $F_\phi(s, c)$ via bilinear interpolation over (s, c) using F .
6. Update the kernel:

$$K^l(x, x') = \sigma_b^2 + \sigma_w^2 F_\phi(s, c).$$

This strategy reduces the overall complexity to

$$\mathcal{O}(n_g^2 n_v n_c + L(n_{\text{train}}^2 + n_{\text{train}} n_{\text{test}})),$$

where the costly Gaussian integrals are computed once, and subsequent layers require only interpolation and matrix updates.

2.4.1 Computing the diagonal terms

Note that, when $x = x'$, we have $c_j = 1$ and hence we cannot update the diagonal elements of K using the lookup formula used in equation 12. In this case, we update the diagonal elements by using a linear interpolation and the following formula.

$$F_i = \frac{\sum_a \phi(u_a)^2 \exp(-u_a^2 / 2s_i^2)}{\sum_a \phi(u_a)^2 \exp(-u_a^2 / 2s_i^2)} \quad (13)$$

The full kernel building and training process is provided in Appendix 3.

2.5 Uncertainty quantification

A key advantage of Gaussian processes is that they naturally provide uncertainty estimates for all predictions (Equation 11). In contrast, estimating uncertainty in standard neural networks is difficult. In the NNGP framework, each test input is assigned a predictive variance (Equation 11).

3 Algorithm

Algorithm 1 Neural Network Gaussian Process (NNGP) Inference

Require: Training data $\mathcal{D} = \{(x_i, t_i)\}_{i=1}^n$, test inputs X_* , depth L , activation ϕ , hyperparameters σ_w^2, σ_b^2 , noise σ_ϵ^2

1: **Normalize inputs:** ensure $\|x_i\|$ is constant for all i

2: **Initialize kernel:**

$$K^0(x, x') = \sigma_b^2 + \sigma_w^2 \frac{x^\top x'}{d_{\text{in}}}$$

3: **for** $l = 1$ to L **do**

4: **for** all pairs (x, x') **do**

5: Define covariance matrix

$$\Sigma = \begin{pmatrix} K^{l-1}(x, x) & K^{l-1}(x, x') \\ K^{l-1}(x', x) & K^{l-1}(x', x') \end{pmatrix}$$

6: Compute kernel update

$$K^l(x, x') = \sigma_b^2 + \sigma_w^2 \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \Sigma)} [\phi(u)\phi(v)]$$

7: **If** analytic form exists (e.g. ReLU), use closed form

8: **Else** approximate expectation numerically via lookup table

9: **end for**

10: **end for**

11: **Construct covariance matrices:**

$$K_{DD} = K^L(X_{\text{train}}, X_{\text{train}}), \quad K_{*D} = K^L(X_*, X_{\text{train}}), \quad K_{**} = K^L(X_*, X_*)$$

12: **Compute predictive mean:**

$$\mu_* = K_{*D}(K_{DD} + \sigma_\epsilon^2 I)^{-1} t$$

13: **Compute predictive covariance:**

$$\Sigma_* = K_{**} - K_{*D}(K_{DD} + \sigma_\epsilon^2 I)^{-1} K_{D*}$$

14: **Classification (optional):** encode labels as zero-mean one-hot vectors and predict

$$\hat{y} = \arg \max_k (\mu_*)_k$$

15: **Hyperparameter selection:** search over $(L, \sigma_w^2, \sigma_b^2, \phi)$ using validation set

16: **return** μ_*, Σ_* , predictions

References

- Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. Deep neural networks as gaussian processes. In *International Conference on Learning Representations (ICLR)*, 2018.
- Radford M. Neal. *Priors for Infinite Networks*, pages 29–53. Springer New York, New York, NY, 1996. doi: 10.1007/978-1-4612-0745-0₂.
- Youngmin Cho and Lawrence K. Saul. Kernel methods for deep learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2009.